

Cisco Configuration Command Reference

Switch, Router & ASA/Firepower CLI — Complete Course Set

ICT2217 Network Security

Hands-on configuration commands, organised by task

Switch Security • ACLs • ASA/Firepower • VRF & SSH • AAA • NTP / Syslog / NetFlow

Contents

A. Switch Security (Layer 2)

MAC table • Port Security • DTP/trunk • native & voice VLAN
STP protection • err-disable recovery • DHCP Snooping • DAI • IP Source Guard

B. Access Control Lists (IOS)

Standard / extended / named • editing • time-based • object-group • anti-recon • verify

C. Cisco ASA / Firepower Firewall

Access & interfaces • security-level • NAT • routing • ACL • MPF • SYN-flood & Slowloris • reset

D. Router DoS / Reconnaissance Defenses (IOS)

Smurf (directed broadcast) • anti-spoofing / established ACLs • ICMP hardening

E. Secure Management

SSH • VRF (both syntaxes) • per-VRF TACACS+

F. AAA

Line passwords • local login • password protection • brute-force block
Authentication • authorization (privilege levels & CLI views) • accounting • test/debug

G. Security Monitoring

Clock • NTP • Syslog • Flexible NetFlow

Note: This reference lists **Cisco device CLI commands only** (switch, router, ASA/Firepower). Server-side setup (AAA / DNS / Syslog / NetFlow-collector / web servers) and attacker tooling are intentionally excluded. Italic *italics* mark values you replace; {a | b} = choose one; [] = optional. Modes: (config) global, (config-if) interface, # privileged EXEC.

A | Switch Security (Layer 2)

A1 MAC Address Table — inspection & aging

Observe the CAM table (used when demonstrating MAC flooding) and tune aging.

```
Switch# show mac address-table dynamic [vlan vlan-id]
Switch# show mac address-table count
Switch# clear mac address-table dynamic
Switch(config)# mac address-table aging-time seconds [vlan vlan-id]    ! default 300; range 10-1000
Switch(config)# mac address-table aging-time 0                        ! disable aging
Switch(config)# no mac address-table aging-time                        ! back to default
```

A2 Port Security — MAC flooding defense

Enable on an ACCESS port; combine maximum / sticky / static as needed.

```
Switch(config)# interface interface-id
Switch(config-if)# switchport mode access
Switch(config-if)# switchport port-security
Switch(config-if)# switchport port-security maximum value
Switch(config-if)# switchport port-security mac-address sticky
Switch(config-if)# switchport port-security mac-address mac-address    ! static entry
Switch(config-if)# switchport port-security violation {protect | restrict | shutdown} ! default
```

Verify:

```
Switch# show port-security
Switch# show port-security interface interface-id
Switch# show port-security address
```

A3 Switch Spoofing defense — disable DTP, set mode explicitly

Manually pin each port to access or trunk and turn off DTP negotiation.

```
Switch(config-if)# switchport mode access    ! or 'switchport mode trunk'
Switch(config-if)# switchport access vlan vlan-id
Switch(config-if)# switchport nonegotiate    ! disable DTP
! Harden unused ports:
Switch(config)# interface range range
Switch(config-if)# switchport access vlan 100    ! park in an unused VLAN
Switch(config-if)# shutdown
Switch(config-if)# switchport trunk allowed vlan remove 100    ! keep unused VLAN off the trunk
```

A4 Double-Tagging defense & Voice VLAN

Give trunks a dedicated native VLAN (unused by any access port), or force native tagging.

```
Switch(config-if)# switchport trunk native vlan vlan-id    ! dedicated native VLAN, e.g. 500
Switch(config)# vlan dot1q tag native                      ! force-tag all trunk traffic (incl. native)
! Voice VLAN (data + auxiliary voice) on an access port:
Switch(config-if)# switchport mode access
Switch(config-if)# switchport access vlan vlan-id          ! data VLAN
Switch(config-if)# switchport voice vlan vlan-id           ! voice VLAN
```

A5 STP protection — BPDU Guard, PortFast, Root Guard

Protect against rogue-root / BPDU-injection attacks.

```

! Per-interface (ports facing user PCs):
Switch(config-if)# spanning-tree portfast
Switch(config-if)# spanning-tree bpduguard enable
! Global default on all PortFast ports:
Switch(config)# spanning-tree portfast bpduguard default
! Root Guard (ports that must stay designated, e.g. toward access switches):
Switch(config-if)# spanning-tree guard root
! Set/lower root priority (multiple of 4096):
Switch(config)# spanning-tree vlan vlan-id priority 24576

```

A6 Recovering err-disabled ports

Ports err-disabled by Port Security / BPDU Guard / DHCP snooping / DAI.

```

! Manual recovery:
Switch(config-if)# shutdown
Switch(config-if)# no shutdown
! Auto recovery:
Switch(config)# errdisable recovery cause reason      ! e.g. psecure-violation, bpduguard
Switch(config)# errdisable recovery interval seconds

```

A7 DHCP Snooping — DHCP starvation / rogue-server defense

Enable globally & per VLAN; trust only the authorized-server port; rate-limit untrusted ports.

```

Switch(config)# ip dhcp snooping
Switch(config)# ip dhcp snooping vlan vlan-id [vlan-id]
Switch(config)# ip dhcp snooping verify mac-address      ! frame MAC must match DHCP chaddr
Switch(config-if)# ip dhcp snooping trust                ! authorized-server / uplink port
Switch(config-if)# ip dhcp snooping limit rate rate      ! 1-2048 pkts/s (untrusted ports)
! Option-82 handling for a downstream relay/router:
Router(config)# ip dhcp relay information trust-all
Router(config-if)# ip dhcp relay information trusted
Switch(config)# no ip dhcp snooping information option    ! disable option-82 insertion

```

Verify:

```

Switch# show ip dhcp snooping
Switch# show ip dhcp snooping binding

```

A8 Dynamic ARP Inspection (DAI) — ARP poisoning / MitM defense

Runs on a DHCP-snooping switch; validates ARP against the binding table on untrusted ports.

```

Switch(config)# ip arp inspection vlan vlan-id
Switch(config)# ip arp inspection validate {src-mac | dst-mac | ip} ! optional extra checks
Switch(config-if)# ip arp inspection trust                          ! trusted uplink ports
Switch(config-if)# ip arp inspection limit {rate pps [burst interval seconds] | none} ! default
! Non-DHCP environment – static ARP ACL:
Switch(config)# arp access-list arp-acl-name
Switch(config-arp-nacl)# permit ip host sender-ip mac host sender-mac
Switch(config)# ip arp inspection filter arp-acl-name vlan vlan-range [static]

```

Verify:

```

Switch# show ip arp inspection
Switch# show ip arp inspection interfaces

```

A9 IP Source Guard (IPSG) — IP/MAC spoofing defense

Permits only source addresses in the binding table; add 'mac-check' + Port Security for MAC protection.

```
Switch(config-if)# ip verify source          ! IP spoofing protection
Switch(config-if)# ip verify source mac-check ! IP + MAC (requires Port Security)
! Static binding (no DHCP snooping):
Switch(config)# ip source binding mac-addr vlan vlan-id ip-addr interface int-id
Switch# show ip verify source
```

B | Access Control Lists (IOS)

Standard numbered: **1–99, 1300–1999** (source IP only, place near destination). Extended numbered: **100–199, 2000–2699** (place near source). Wildcard mask = inverse of subnet mask; **host** = one address, **any** = all.

B1 Standard numbered ACL

```
R(config)# access-list 1 remark text ! optional comment
R(config)# access-list 1 permit 192.168.10.0 0.0.0.255
R(config)# access-list 1 permit host 192.168.20.3
R(config)# access-list 1 deny any
R(config-if)# ip access-group 1 {in | out}
```

B2 Standard named ACL

```
R(config)# ip access-list standard NAME
R(config-std-nacl)# permit host 192.168.20.3
R(config-std-nacl)# permit 192.168.40.0 0.0.0.255
R(config-if)# ip access-group NAME {in | out}
```

B3 Extended numbered ACL (protocol / ports)

Operators before ports: eq, gt, lt, neq, range. Ports as numbers or names (eq 80 / eq www). ICMP uses message-type keywords (e.g. echo), not eq.

```
R(config)# access-list 100 remark text
R(config)# access-list 100 deny tcp 10.3.0.0 0.0.255.255 host 10.1.0.1 eq 23 log
R(config)# access-list 100 deny icmp any host 192.168.0.101 echo
R(config)# access-list 100 permit tcp 192.168.10.0 0.0.0.255 any eq 443
R(config)# access-list 100 permit ip any any
R(config-if)# ip access-group 100 {in | out}
```

B4 Extended named ACL

```
R(config)# ip access-list extended NAME
R(config-ext-nacl)# permit tcp 192.168.20.0 0.0.0.255 host 10.1.1.1 eq 443
R(config-ext-nacl)# deny ip 192.168.50.0 0.0.0.255 any
R(config-ext-nacl)# permit ip any any
R(config-if)# ip access-group NAME {in | out}
```

B5 Editing an ACL by sequence number

Newer IOS auto-inserts sequence numbers (step 10). Insert / delete individual ACEs in place.

```
R(config)# ip access-list extended NAME ! (or 'ip access-list extended 100')
R(config-ext-nacl)# 30 permit ip 192.168.10.0 0.0.0.255 192.168.20.0 0.0.0.255 ! insert at 30
R(config-ext-nacl)# no 30 ! delete line 30
R(config)# ip access-list resequence NAME start-number increment
```

B6 Time-based ACL

Activate an ACE only during set times (router clock; sync with NTP).

```
R(config)# time-range NAME
R(config-time-range)# periodic weekdays 08:00 to 17:00 ! or 'absolute start ... end ...'
R(config)# access-list 100 permit tcp any any eq 80 time-range NAME
```

B7 Object-group ACL

Group networks and services for readable, reusable ACLs.

```
R(config)# object-group network Inside_Networks
R(config-network-group)# 10.0.0.0 /24
R(config-network-group)# host 10.0.1.1
R(config-network-group)# group-object OTHER-GROUP          ! nesting
R(config)# object-group service Host_Services
R(config-service-group)# tcp www
R(config-service-group)# tcp 443
R(config)# access-list 100 permit object-group Host_Services object-group Inside_Networks any
R(config-if)# ip access-group 100 {in | out}
```

B8 Anti-reconnaissance / anti-spoofing ACLs

Permit only return traffic; match specific TCP flags; stop ICMP information leaks.

```
! Allow return traffic only (ACK/RST/SYN-ACK/FIN):
R(config-ext-nacl)# permit tcp any 200.1.1.0 0.0.0.255 established
! Match specific TCP flags:
R(config-ext-nacl)# permit tcp any any match-all +ack
R(config-ext-nacl)# permit tcp any any match-all +syn +ack
! Do not leak ICMP unreachable when dropping scans:
R(config-if)# no ip unreachable
```

B9 ACL verification

```
R# show access-lists [number | name]
R# show ip interface interface-id          ! shows inbound/outbound ACL applied
R# clear access-list counters [number | name]
```

C | Cisco ASA / Firepower Firewall

Firepower running the ASA image. ASA ACLs use a **subnet mask** (not a wildcard). Higher security-level → lower is allowed by default; return traffic is stateful.

C1 Access & interface as routed port

Enter privileged/global config; convert a Firepower switchport to a routed port (as on an L3 switch).

```
ciscoasa> enable
ciscoasa# configure terminal
ciscoasa(config)# interface interface-id
ciscoasa(config-if)# no switchport           ! make it a routed port (e.g. Ethernet1/1)
```

C2 Security zones — nameif, security-level, IP

Each active interface needs a name, a security-level (0=Internet, 100=inside, ~50=DMZ) and an IP.

```
ciscoasa(config)# interface interface-id
ciscoasa(config-if)# nameif interface-name
ciscoasa(config-if)# security-level 0-100
ciscoasa(config-if)# ip address ip-addr subnet-mask
! Allow traffic between two same-level interfaces:
ciscoasa(config)# same-security-traffic permit inter-interface
```

C3 Remove pre-configured NAT / name (lab)

Outside interface ships with dynamic NAT; 'inside' name may be pre-bound to VLAN 1.

```
ciscoasa(config)# object network obj_any
ciscoasa(config-network-object)# no nat (any,outside) dynamic interface
ciscoasa(config)# interface vlan 1
ciscoasa(config-if)# no nameif inside
```

C4 Static route

```
ciscoasa(config)# route int-name subnet-block netmask next-hop-ip
```

C5 ACL to override default policy (e.g. Internet → DMZ server)

Note the subnet-mask style and application with access-group.

```
ciscoasa(config)# access-list acl-name extended permit tcp any host 209.165.200.226 eq www
ciscoasa(config)# access-group acl-name in interface int-name
```

C6 Modular Policy Framework (MPF)

class-map (match traffic) → policy-map (action) → service-policy (apply globally or per interface).

```
ciscoasa(config)# class-map class_map_name
ciscoasa(config-cmap)# match {access-list acl | default-inspection-traffic | ...}
ciscoasa(config)# policy-map policy_map_name
ciscoasa(config-pmap)# class class_map_name
ciscoasa(config-pmap-c)# inspect engine-name           ! e.g. inspect icmp
ciscoasa(config)# service-policy policy_map_name {global | interface int-name}
```


Enable ICMP inspection in the default global policy (so ping works):

```
ciscoasa(config)# policy-map global_policy
ciscoasa(config-pmap)# class inspection_default
ciscoasa(config-pmap-c)# inspect icmp          ! allow ping through the firewall
```

C7 TCP SYN flooding defense — embryonic limits (TCP Intercept)

Set embryonic-connection limits via MPF; TCP Intercept + SYN cookies activate when crossed. limit = 0–2000000 (0 = unlimited).

```
ciscoasa(config)# access-list acl-name extended permit ...
ciscoasa(config)# class-map class_map_name
ciscoasa(config-cmap)# match access-list acl-name
ciscoasa(config)# policy-map policy_map_name
ciscoasa(config-pmap)# class class_map_name
ciscoasa(config-pmap-c)# set connection embryonic-conn-max limit
ciscoasa(config-pmap-c)# set connection per-client-embryonic-max limit
! (remember: service-policy to apply the policy-map)
ciscoasa(config)# threat-detection statistics tcp-intercept
ciscoasa# show threat-detection statistics top tcp-intercept
```

C8 Slowloris defense — connection limits

Same MPF workflow; tune 'set connection' parameters (per-client / conn-max / embryonic / timeout).

```
ciscoasa(config-pmap-c)# set connection ?          ! explore parameters, e.g.:
ciscoasa(config-pmap-c)# set connection conn-max limit
ciscoasa(config-pmap-c)# set connection per-client-max limit
! (apply with a class-map + service-policy on the required interface)
```

C9 State table, factory reset & reload

```
ciscoasa# show conn                                ! state / connection table
ciscoasa(config)# configure factory-default        ! restore factory defaults
ciscoasa# write erase                               ! erase startup-config
ciscoasa# reload
```

D | Router DoS / Reconnaissance Defenses (IOS)

D1 Smurf attack — lab enablement vs. defense

Directed broadcast is disabled by default. The first two commands re-enable it ONLY to demonstrate the attack in the lab; production routers should leave it disabled.

```
! Lab-only: enable directed broadcast to reproduce Smurf
R(config-if)# ip network-broadcast          ! ingress interface: accept directed broadcast
R(config-if)# ip directed-broadcast        ! egress interface: broadcast → MAC broadcast
! Defense: leave directed broadcast disabled + BCP-38 anti-spoofing ACLs (see B8)
```

D2 Anti-spoofing / return-traffic ACLs & ICMP hardening

BCP 38 / RFC 2827: filter packets whose source IP is outside the allocated block; harden ICMP.

```
R(config-ext-nacl)# permit tcp any local-block wildcard established ! return traffic only
R(config)# ip access-list extended anti-spoof
R(config-ext-nacl)# deny ip bogus-source-block wildcard any
R(config-ext-nacl)# permit ip any any
R(config-if)# no ip unreachable                                     ! do not leak ICMP unreachable
```

E | Secure Management

E1 SSH for remote access

Hostname + domain name feed the RSA key; use SSH v2 and login local on the VTY lines.

```
R(config)# hostname host-name
R(config)# ip domain-name domain-name      ! (some IOS: 'ip domain name')
R(config)# crypto key generate rsa modulus 2048      ! or 4096
R(config)# ip ssh version 2
R(config)# ip ssh authentication-retries number
R(config)# ip ssh timeout seconds
R(config)# line vty 0 15
R(config-line)# login local
R(config-line)# transport input ssh
R# show ip ssh ; show ssh
```

E2 VRF — create, assign, verify

Assigning an interface to a VRF deletes its existing IPv4 address, so re-add the IP afterward.

```
! Newer syntax (IOS XE / Catalyst 8200):
R(config)# vrf definition vrf-name
R(config-vrf)# address-family ipv4
R(config-if)# vrf forwarding vrf-name
R(config-if)# ip address ip-addr subnet-mask
! Older IOS syntax:
R(config)# ip vrf vrf-name
R(config-if)# ip vrf forwarding vrf-name
R(config)# ip route vrf vrf-name prefix mask next-hop
! VRF-aware verification / testing:
R# show ip route vrf vrf-name      ! ('vrf *' shows all)
R# ping vrf vrf-name target-ip
R# show run vrf [vrf-name]
```

E3 Per-VRF TACACS+ (management VRF)

Point AAA at a server reachable only through a VRF, sourced from the VRF interface.

```
R(config)# aaa group server tacacs+ management
R(config-sg-tacacs+)# server-private ip-addr key key
R(config-sg-tacacs+)# ip vrf forwarding vrf-name
R(config-sg-tacacs+)# ip tacacs source-interface interface-id
R(config)# aaa authentication login default group management local
R# test aaa group management user password new-code      ! verify (use 'new-code')
R# debug tacacs ; debug aaa authentication                ! troubleshoot
```

F | AAA (Authentication, Authorization, Accounting)

F1 Simple line password

Shared password on console / VTY (weakest). enable password for privileged mode.

```
Switch(config)# enable password password-string
Switch(config)# line con 0
Switch(config-line)# password password-string
Switch(config-line)# login
Switch(config)# line vty 0 15
Switch(config-line)# password password-string
Switch(config-line)# login
```

F2 Local username login (required for SSH)

```
Switch(config)# username name secret password
Switch(config)# line vty 0 15
Switch(config-line)# login local
Switch(config-line)# transport input ssh
```

F3 Password protection

'secret' stores an MD5 hash (vs. clear-text 'password'); sha256/scrypt are stronger.

```
Switch(config)# service password-encryption          ! weak, obfuscates clear-text passwords
Switch(config)# enable secret password-string        ! MD5 (Type 5)
Switch(config)# username name secret password
! Stronger algorithms where supported:
Switch(config)# enable algorithm-type {sha256 | scrypt} secret password-string
```

F4 Login brute-force protection

Block-for = quiet-mode lockout; permit admins during lockout via an ACL; log attempts.

```
R(config)# login block-for seconds attempts tries within seconds
R(config)# login delay seconds
R(config)# login quiet-mode access-class {acl-name | acl-number}
R(config)# login on-success log
R(config)# login on-failure log
R# show login failures
```

F5 Authentication via TACACS+ / RADIUS (method lists)

Enable AAA, define server(s), build a method list. Default list auto-applies to console/VTY; a custom list must be applied to a line. List several methods for fallback.

```

R(config)# aaa new-model
! TACACS+ server:
R(config)# tacacs server NAME
R(config-server-tacacs)# address ipv4 ip-addr
R(config-server-tacacs)# key shared-secret
! RADIUS server:
R(config)# radius server NAME
R(config-radius-server)# address ipv4 ip-addr auth-port 1812 acct-port 1813
R(config-radius-server)# key shared-secret
! Method list (default or custom):
R(config)# aaa authentication login default group tacacs+ group radius local
R(config)# aaa authentication login MY-LIST group tacacs+ local
R(config-line)# login authentication MY-LIST ! apply a custom list to the line

```

Method keywords: **local**, **local-case**, **enable**, **none**, **group tacacs+**, **group radius**. Keep **local** last as a fallback so you are not locked out if the servers are unreachable.

F6 Authorization — privilege levels

Move commands to custom levels 2–14; assign levels to users; authorize via AAA.

```

R(config)# privilege exec level level command-string ! e.g. privilege exec level 5 ping
R(config)# enable secret level level password ! e.g. enable secret level 5 cisco5
R(config)# privilege exec reset command-string ! reset to default level
R(config)# username name privilege level secret password
R(config)# aaa authorization exec default group tacacs+ group radius local
R> show privilege

```

F7 Authorization — role-based CLI views & superviews

Enable AAA, enter root view, then define CLI views (max 15). Superviews group CLI views.

```

R(config)# aaa new-model
R# enable view ! root view (uses enable secret)
R(config)# parser view VIEWNAME
R(config-view)# secret view-password
R(config-view)# commands exec include show version
R(config-view)# commands exec include ping
R(config-view)# commands parser-mode {include | include-exclusive | exclude} [all] command
! Superview:
R(config)# parser view SUPERNAME superview
R(config-view)# secret password
R(config-view)# view VIEWNAME
! Assign a view to a user:
R(config)# username name view VIEWNAME secret password
R# enable view VIEWNAME ; show parser view [all]

```

F8 Accounting

exec = session start/stop; commands (TACACS+ only) = per-privilege-level command logging.

```

R(config)# aaa accounting exec default start-stop group tacacs+ group radius
R(config)# aaa accounting commands 15 default start-stop group tacacs+

```

G | Security Monitoring

G1 Device clock

Set time before configuring NTP; timezone is UTC-relative (e.g. SGT = UTC+8).

```
R# clock set hh:mm:ss day month year          ! e.g. clock set 9:39:00 05 july 2013
R(config)# clock timezone name hours-offset   ! e.g. clock timezone SGT 8
R# show clock
```

G2 NTP — modes, authentication, ACL

A device that syncs to a source automatically becomes a server for others.

```
R(config)# ntp master [stratum]                ! internal master
R(config)# ntp server {ip | hostname} [version n] [prefer] [key key-number]
R(config)# ntp peer {ip | hostname} [version n] [prefer]
R(config-if)# ntp broadcast [version n]        ! server side
R(config-if)# ntp broadcast client             ! client side
! Authentication (MD5):
R(config)# ntp authentication-key key-number md5 string
R(config)# ntp trusted-key key-number
R(config)# ntp authenticate
R(config)# ntp server ip key key-number
! Protection & logging:
R(config)# ntp access-group {peer | serve | serve-only | query-only} acl-number
R(config)# ntp logging
R(config)# ntp server vrf Mgmt-vrf ip          ! over 00B/VRF
R# show ntp status ; show ntp associations
```

G3 Syslog

Severity 0 (emergencies) – 7 (debugging); a level logs it and all lower (more severe) levels. Default transport UDP 514.

```
R(config)# service timestamps log datetime msec
R(config)# logging buffered [size] [severity]    ! log to RAM
R(config)# logging host ip-addr                 ! (or 'logging ip-addr')
R(config)# logging trap {level-name | 0-7}      ! e.g. logging trap warnings / 4
R(config)# logging monitor                     ! enable over Telnet/SSH
R(config)# no logging console                   ! stop console spam
R(config-line)# logging synchronous             ! don't interrupt command entry
R# terminal monitor                             ! show logs on a vty session
R# show logging
```

G4 Flexible NetFlow

Four steps: flow record → flow exporter → flow monitor → apply to interface.

```
! 1) Flow record (what to collect)
R(config)# flow record REC
R(config-flow-record)# match ipv4 source address
R(config-flow-record)# match ipv4 destination address
R(config-flow-record)# match transport source-port
R(config-flow-record)# match transport destination-port
R(config-flow-record)# match ipv4 protocol
R(config-flow-record)# collect counter bytes
R(config-flow-record)# collect counter packets
! 2) Flow exporter (where to send)
R(config)# flow exporter EXP
R(config-flow-exporter)# destination collector-ip
R(config-flow-exporter)# transport udp port
R(config-flow-exporter)# source loopback0
R(config-flow-exporter)# export-protocol netflow-v9
! 3) Flow monitor (bind record + exporter)
R(config)# flow monitor MON
R(config-flow-monitor)# record {REC | netflow ipv4 original-input}
R(config-flow-monitor)# exporter EXP
R(config-flow-monitor)# cache timeout active seconds
! 4) Apply to interface
R(config-if)# ip flow monitor MON {input | output}
```

Verify:

```
R# show flow record REC
R# show flow exporter EXP
R# show flow monitor [MON [statistics | cache]]
```